TRANSFORMED 2 SUCCEED

transformed2succeed.com

FREE LIVE SESSION · 30 MINUTES

Zero to AI-ML Systems Engineering

No experience needed. Write a fraud rule, teach a machine to beat it, then serve it as a tested API.

PRESENTER Emmanuel N. **LIVE** Mondays & Thursdays · 7:30 PM Central · 30 min

TONIGHT'S 30 MINUTES

Welcome! Here's the plan

0–2 min

Welcome

Say hi in the chat: your name, where you're joining from, and your role.

2–20 min

Three modules

One real idea from each module of Zero to AI-ML Systems Engineering, with the lab behind it.

20–23 min

Keep going

How to get the full course, every lab, and support along the way.

23–30 min

Your questions

Live Q&A. Drop questions in the chat any time; I'll answer at the end.

WHY THIS COURSE MATTERS

Three phrases take you from zero to a working ML service

01

Data in, decision out

Every program has this shape. In Module 1 you write the rule yourself.

02

Never grade on study pile

Module 2: a machine learns the rule from examples, and you judge it honestly.

03

Save it, serve it, test it

Module 3: the model becomes a file, an API anyone can call, and a robot checks it.

Three modules, one fraud detector, built from zero

MODULE 1

Talk to the Machine

Use a terminal, learn Python's five building blocks, test a rule on 5,000 payments.

MODULE 2

Teach a Machine the Rule

Train a decision tree, catch it memorising, judge it with recall and a dial.

MODULE 3

Put Your Model to Work

Track every run, crown a champion, serve it with Flask, test it with pytest.

01

MODULE 1 OF 3

Talk to the Machine

No experience needed. Not one line of code. Data in, decision out.

Every program is data in, decision out

01

Data in

What the program receives: a payment of $1,850 at 3 a.m.

02

The rule

The decision you write down: amount > 1000 and hour < 6.

03

Decision out

What the program answers: FRAUD or LEGIT.

If you can say the rule in English, you can write it in Python.

Five shapes. Everything else is combinations.

01 Variable

name = "Sam". A labelled box holding a value.

02 List

[p1, p2, p3]. A row of boxes: many payments.

03 Dictionary

{"amount": 1850, "hour": 3}. Labels pointing to values: one payment.

04 Loop

for p in payments: do the same thing to each one.

05 Function

def is_fraud(p): a named, reusable decision.

English in, Python out

ENGLISH

over $1,000 and before 6 a.m.

Say the rule out loud first. If you can say it clearly, you're halfway there.

VS

PYTHON

amount > 1000 and hour < 6

```
return payment["amount"] > 1000 and
payment["hour"] < 6
```

Look up a value, compare it, combine with and, return True or False. That's the whole rule.

Run your rule, then test it on 5,000 payments

TERMINAL

```
$ python3 module-1/hello.py
$ python3 module-1/rule.py
$ python3 module-1/explore.py
```

hello.py
your first program: edit, save, run

rule.py
five payments, one is_fraud function, five verdicts

explore.py
the four-question ritual, then your rule on every row

the ritual
shape · head · info · describe, before every dataset

YOU SHOULD SEE Your rule caught 61 of 137 frauds and raised 59 false alarms.

How good is a hand-written rule, really?

5,000

pretend card payments in the dataset

137

of them are real fraud

61

frauds your rule caught: less than half

59

false alarms: innocent people accused

Caught, missed, false alarms. Not just 'it works'. A hand-written rule hits a ceiling.

02

MODULE 2 OF 3

Teach a Machine the Rule

Show it examples. Then judge it honestly. Never grade on the study pile.

Who writes the rule?

PROGRAMMING

You write it

Data + your rule → answers. Module 1: 61 of 137 caught. You hit a ceiling.

VS

MACHINE LEARNING

It finds it

Data + answers → the rule. A decision tree writes its own flowchart of yes/no questions.

Same shape as Module 1: data in, decision out. The only change is who writes the middle box.

Split · choose · train · predict · score

1

Split

train_test_split: 80% study pile, 20% quiz pile, locked away.

2

Choose + train

DecisionTreeClassifier(). fit(X_train, y_train) on the study pile only.

3

Predict

model.predict(X_test): answers for payments it never saw.

4

Score

accuracy_score(y_test, predictions): compare with the real answers.

Same data, two trees

100.0%

no limit · study pile (20 levels, 160 leaves)

95.8%

no limit · quiz pile: it memorised

98.0%

max_depth=3 · quiz pile: simpler, and better

The gap between study and quiz is the diagnosis. Big gap: memorising (overfitting). Small gap: learning.



“amount > 1412 and hour ≤ 6 → FRAUD”

Your Module 1 rule was amount > 1000 and hour < 6. The machine found nearly the same rule, by itself, from examples.

A model that does nothing scores 97.3%

1,000

payments in the quiz pile

27

of them are fraud

97.3%

accuracy of 'always say LEGIT'

0

frauds it catches

Accuracy hides rare events. Ask two questions instead: recall (how much fraud did we catch?) and precision.

Train it, judge it, then turn the dial

TERMINAL

```
$ python3 module-2/train.py --max_depth 3
$ python3 module-2/judge.py
$ python3 module-2/judge.py --threshold 0.2
```

`--max_depth 3`
how many questions deep the tree may go

`judge.py`
the 'always LEGIT' trap, then the four boxes

`--threshold 0.5`
8 of 27 caught · 6 false alarms · recall 30%

`--threshold 0.2`
18 of 27 caught · 35 false alarms · recall 67%

YOU SHOULD SEE 0.2 → caught 18 · false alarms 35 · recall 66.7%

03

MODULE 3 OF 3

Put Your Model to Work

From a script on your laptop to a service anyone can call. Save it, serve it, test it.

The gap between an experiment and a service

01

Save it

A trained model is just a file. joblib writes it to disk; load it anywhere.

02

Serve it

An API wraps the file so any app can send a payment and get an answer.

03

Test it

pytest checks the API for you, every time you change anything. In seconds.

Every production ML system on earth is a bigger version of these three boxes.

Write the business rule down first

THE LAB NOTEBOOK

One line per run in runs.jsonl

Settings, scores and the model file. Memory forgets. The notebook doesn't.

VS

THE RULE

precision ≥ 0.30, then max recall

Run 2 wins: recall 67%, precision 34%. Copied to current.joblib, the champion.

Run 4, a deeper tree, did worse. Without the notebook, you'd never know.

Six words that cover every API

01 **GET /health**

Are you alive? Which model are you running?

02 **POST /predict**

Here's a payment. Fraud or not?

03 **JSON**

The note's format: {"amount": 2500}. Just like a Python dictionary.

04 **200**

OK: here's your answer.

05 **400**

Your note isn't JSON.

06 **422**

JSON, but a field is wrong, like hour 99. The model never sees it.

Serve it, then knock on the door

TERMINAL

```
$ python3 module-3/api.py        # terminal 1
$ curl -X POST localhost:5000/predict \
  -H "Content-Type: application/json" \
  -d '{"amount": 2500, "hour": 3, "new_device": 1}'
```

`api.py`
Flask app: GET /health and POST /predict, port 5000

`curl`
knock on a door from a second terminal

`-d '{...}'`
the note: a big night-time payment, new device

`"hour": 99`
refused with 422; the model never sees it

YOU SHOULD SEE {"fraud_score":1.0,"label":"FRAUD","model_run":2}

Five checks, two seconds, every time

TERMINAL

```
$ python3 -m pytest module-3/tests -q
```

`assert`
a sentence that must be true, or the test fails

`5 tests`
health · fraud · legit · bad hour · missing field

`break it on purpose`
misspell FRAUD in api.py, rerun: 1 failed

`change, test, ship`
the loop for the rest of your career

YOU SHOULD SEE ..... 5 passed (after the typo: 1 failed, 4 passed)

What you take from three modules

Data in, decision out

Module 1: terminal, five building blocks, a rule tested on 5,000 payments.

Never grade on the study pile

Module 2: split, catch overfitting, judge with recall, precision and a dial.

Save it, serve it, test it

Module 3: notebook, champion, a Flask API, and pytest checking it.



From zero to a tested ML service. You can do this.

Next in the full course: Terraform to the cloud, CI/CD with drift monitoring and a promotion gate, then an AI assistant.

CODE ALONG

Set up your lab before the next session

1

Pick where to run it

Easiest: GitHub Codespaces. Or Windows, or Mac.

2

Get the lab files

Download zero-2-ai-foundations-lab.zip from the repo's downloads folder, then unzip it.

3

Install and check

Follow SETUP.md step by step. The check tells you when you're ready.

4

Run one test per module

SETUP.md has one quick command for each of the 3 modules.

Everything is here: **github.com/Here2ServeU/live-zero-to-ai-ml** · slides, setup and labs

THE MEMBERSHIP

Every course. One price.

$49

per month

All 8 courses, every module and every lab, plus a live session with me every month.

ALL 8 COURSES INCLUDED

- Linux for AI-ML Systems
- Python for AI-ML
- **Zero to AI-ML Systems Engineering**
- DevOps for AI-ML Systems
- SRE for AI-ML Systems
- GPU Optimization for AI-ML Systems
- FinOps for AI-ML Systems
- AI Agent Operations

BUILD WITH ME, LIVE

Once a month, we build something together

1

Learn at your pace

Work through any course, any module, on your own schedule.

2

Bring a real problem

An idea, a project at work, or a lab you want to take further.

3

Meet me live

Once a month we get on a call and build it together, step by step.

4

Leave with it built

Something real you made, understand and can show.

FREE LIVE SESSIONS

Every Monday and Thursday, 7:30 PM Central

A different course each session. The full rotation takes 4 weeks, then starts again.

	WEEK 1	WEEK 2	WEEK 3	WEEK 4
MONDAY	Linux for AI-ML Systems	TONIGHT Zero to AI-ML Systems Engineering	SRE for AI-ML Systems	FinOps for AI-ML Systems
THURSDAY	Python for AI-ML	NEXT DevOps for AI-ML Systems	GPU Optimization for AI-ML Systems	AI Agent Operations

Join the membership today

$49/month · all 8 courses · a live build session with me every month

transformed2succeed.com

Next free session: Thursday, 7:30 PM Central · DevOps for AI-ML Systems

Questions? Let's take them now.

Emmanuel N. · Transformed 2 Succeed